

Exposé – Abschlussarbeit IKT-Master

Konzeption und Implementierung eines LLM-Tutors für digitale Mathematikaufgaben

Sidney Göhler

21. Juli 2026

Zusammenfassung

Die Arbeit untersucht die Konzeption und Implementierung eines LLM-basierten Tutors für digitale Mathematikaufgaben. Als technische Ausgangsbasis dienen Moodle-STACK-Aufgaben aus dem Themenbereich Differentialrechnung. STACK und das zugrunde liegende Computer-Algebra-System Maxima übernehmen die symbolische Bewertung mathematischer Eingaben, während ein selbst gehostetes Large Language Model adaptive didaktische Hinweise generiert. Die Tutorsteuerung erfolgt über einen expliziten Prompt-Workflow mit gestuften Hilfen und Regeln zur Vermeidung unmittelbaren Lösungs verrats. Zusätzlich wird untersucht, in welcher Form Maxima beziehungsweise eine Headless-STACK-Instanz als mathematisches Prüfwerkzeug oder als aufrufbares Werkzeug eines LLM-Agenten eingebunden werden kann. Die technische Orchestrierung soll prototypisch über eine Workflow-Plattform wie n8n erfolgen. Der Prototyp wird anhand eines Satzes von Differentialrechnungsaufgaben mit typischen Fehlern evaluiert. Optional ist eine explorative Erprobung mit Studierenden mittels Think-Aloud-Methode vorgesehen.

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1 Ausgangslage und Problemstellung	3
2 Forschungsfrage	5
3 Zielsetzung	7
4 Untersuchungsgegenstand und Eingrenzung	9
5 Methodisches Vorgehen	11
5.1 Literatur- und Marktanalyse	11
5.2 Analyse der Anforderungen	12
5.3 Auswahl und Modellierung des Aufgabensatzes	12
5.4 Entwicklung des technischen Prototyps	13
5.5 Anbindung von Maxima und Headless STACK	14
5.6 Workflow-Orchestrierung mit n8n	15
5.7 Kontext- und Wissensbasis	15
5.8 Prompt-, Workflow- und Tutor-Design	16
5.9 Auswahl eines selbst gehosteten LLMs	17
5.10 Evaluation	18
6 Erwartete Ergebnisse und wissenschaftlicher Beitrag	21
7 Quellenlage / aktueller Stand der Technik	22
7.1 Mathematisches E-Assessment und STACK	22
7.2 LLM-basierte Mathematiktutoren und Hint-Generierung	22
7.3 Retrieval-Augmented Generation und Kontextsteuerung	23
7.4 Symbolische Verifikation und neuro-symbolische Ansätze	23
7.5 Kommerzielle und institutionelle Systeme	23
7.6 Forschungslücke	24
8 Zeitplan	25
9 Vorläufige Gliederung	27

◁ 1 Ausgangslage und Problemstellung

Digitale Übungs- und Prüfungssysteme sind in mathematischen Grundlagenveranstaltungen an Hochschulen fest etabliert. Insbesondere das Open-Source-E-Assessment-System **STACK**, das als Fragetyp in Moodle eingesetzt werden kann, ermöglicht die symbolische Bewertung mathematischer Eingaben auf Basis des Computer-Algebra-Systems Maxima. Dadurch können nicht nur numerische Endergebnisse, sondern auch algebraisch äquivalente Ausdrücke erkannt und bewertet werden. Über sogenannte **Potential Response Trees** (PRTs) können Lehrende typische Fehlermuster modellieren, durch symbolische Tests identifizieren und mit spezifischem Feedback versehen.

Trotz dieser Stärken bleibt das Feedback in STACK-Aufgaben in der Regel vorab definiert. Es muss durch Lehrende für einzelne Antwort- und Fehlerfälle formuliert werden und kann nur begrenzt flexibel auf individuelle Lösungswege, Ausdrucksweisen oder Verständnisschwierigkeiten reagieren. Aktuelle Large Language Models (LLMs) eröffnen demgegenüber neue Möglichkeiten für adaptive, sprachlich differenzierte und dialogorientierte Lernunterstützung. LLMs können Erklärungen generieren, aktivierende Rückfragen formulieren und Hinweise an den jeweiligen Bearbeitungsstand anpassen.

Der Einsatz von LLMs in mathematischen Lernumgebungen ist jedoch mit spezifischen Risiken verbunden. Dazu zählen insbesondere fachlich fehlerhafte Aussagen, Halluzinationen, unzuverlässige mathematische Bewertungen und die unmittelbare Ausgabe vollständiger Lösungen. Ein LLM sollte daher nicht als alleinige mathematische Bewertungsinstanz eingesetzt werden. Stattdessen erscheint eine neuro-symbolische Aufgabenteilung sinnvoll: Ein deterministisches mathematisches System prüft die formale Korrektheit, während das LLM die didaktische und sprachliche Aufbereitung übernimmt.

Die geplante Arbeit setzt an dieser Schnittstelle an. STACK und Maxima sollen die mathematische Bewertung und gegebenenfalls Fehlerdiagnose übernehmen. Ein externes, selbst gehostetes LLM soll auf Basis der Aufgabeninformationen, der Studierendenantwort und verfügbarer Prüfergebnisse adaptive didaktische Hinweise generieren. Untersucht wird dabei auch, ob Maxima oder eine Headless-STACK-Instanz als aufrufbares Werkzeug in den

◀1 Ausgangslage und Problemstellung

Tutor-Workflow integriert werden kann. Eine solche Anbindung könnte über eine definierte Schnittstelle oder agentenähnliche Werkzeugnutzung erfolgen und zur Prüfung mathematischer Ausdrücke, Zwischenschritte oder generierter Tutorhinweise eingesetzt werden.

Die Orchestrierung der einzelnen Komponenten kann prototypisch über eine Workflow-Plattform wie *n8n* realisiert werden. Ein möglicher Workflow umfasst die Entgegennahme von STACK-Daten, die Auswahl aufgabenspezifischer Kontextinformationen, den optionalen Aufruf von Maxima beziehungsweise Headless STACK, die Erzeugung eines strukturierten Prompts, den Aufruf des selbst gehosteten LLMs und die Rückgabe eines Tutorhinweises.

Als fachliche Anwendungsdomäne wird die **Differentialrechnung** gewählt. Damit wird die Untersuchung nicht ausschliesslich auf die Kettenregel oder verkettete Exponentialfunktionen beschränkt, sondern auf einen überschaubaren Satz typischer Ableitungsaufgaben erweitert. Dieser kann beispielsweise Potenz-, Produkt-, Quotienten- und Kettenregel sowie Kombinationen dieser Regeln umfassen.

Eine exemplarische Aufgabe lautet:

$$f(x) = -5e^{x^2 - 2e^x}.$$

Die korrekte Bearbeitung erfordert unter anderem die Anwendung der Kettenregel, die Ableitung des inneren Exponenten und die Berücksichtigung des konstanten Faktors. Typische Fehler können entweder durch einen entsprechend modellierten PRT erkannt oder alternativ durch einen mathematischen Prüfworkflow mit Maxima beziehungsweise Headless STACK analysiert werden. Ein Vergleich beider Ansätze ist als optionale Untersuchung vorgesehen.

◀ 2 Forschungsfrage

Die zentrale Forschungsfrage der Arbeit lautet:

Wie kann ein selbst gehosteter LLM-Tutor für digitale Mathematikaufgaben so konzipiert und implementiert werden, dass er Studierende auf Grundlage symbolischer mathematischer Prüfungen durch adaptive Hinweise unterstützt, ohne die vollständige Lösung unmittelbar vorzugeben?

Aus dieser übergeordneten Forschungsfrage ergeben sich folgende Teilfragen:

- 1. Informationsübergabe und Systemarchitektur**
Welche Informationen müssen aus einer digitalen Mathematikaufgabe an den LLM-Tutor übergeben werden, um aufgaben- und fehlerbezogene Hilfestellungen generieren zu können?
- 2. Symbolische Prüfung und Fehlerdiagnose**
Wie können STACK, Potential Response Trees, Maxima oder eine Headless-STACK-Instanz zur Prüfung von Studierendenantworten und gegebenenfalls zur Absicherung generierter Tutorhinweise eingesetzt werden?
- 3. Vergleich von Diagnoseansätzen**
Welche Unterschiede ergeben sich zwischen einer vorab durch Lehrende modellierten Fehlerdiagnose mittels PRT und einer dynamischen mathematischen Analyse durch Maxima beziehungsweise Headless STACK?
- 4. Prompt- und Workflow-Design**
Wie müssen Prompt, Tutor-Policy und technische Prozesssteuerung gestaltet sein, damit das LLM fachlich korrekte, verständliche und didaktisch angemessene Hinweise erzeugt?
- 5. Vermeidung von Lösungsverrat**
Wie können gestufte Hilfen und ein im Prompt hinterlegter Tutor-Workflow das Risiko reduzieren, dass das LLM unmittelbar die vollständige Lösung ausgibt?

◀2 Forschungsfrage

6. Modellwahl

Welches selbst gehostete LLM eignet sich hinsichtlich mathematischer Zuverlässigkeit, deutschsprachiger Ausdrucksqualität, Befolgung der Tutor-Policy und Antwortzeit für den betrachteten Anwendungsfall?

7. Technische und didaktische Qualität

Wie zuverlässig funktioniert der Prototyp bei einem Satz typischer Differentialrechnungsaufgaben und wie sind die generierten Hinweise hinsichtlich fachlicher Korrektheit, Fehlerbezug, Verständlichkeit, Hilfreichkeit und Lösungsverrat zu bewerten?

◀ 3 Zielsetzung

Ziel der Arbeit ist die Konzeption, prototypische Implementierung und Evaluation eines LLM-Tutors für digitale Mathematikaufgaben aus dem Themenbereich Differentialrechnung.

Der Prototyp soll eine Informationskette zwischen einer digitalen Aufgabe, einem symbolischen mathematischen Prüfsystem und einem selbst gehosteten LLM realisieren. STACK und Maxima übernehmen dabei die formale mathematische Bewertung. Das LLM übernimmt die sprachliche und didaktische Formulierung von Hinweisen. Hierzu sollen Aufgabeninformationen, Studierendenantworten, Prüfergebnisse und gegebenenfalls PRT-Diagnosecodes in einem kontrollierten Workflow verarbeitet werden.

Konkret verfolgt die Arbeit folgende Ziele:

- Analyse wissenschaftlicher Arbeiten und bestehender kommerzieller Systeme zu digitalen Mathematikutoren, LLM-basiertem Feedback, RAG, symbolischer Verifikation und intelligenten Tutorsystemen,
- Entwicklung eines technischen Interfaces zur Übergabe relevanter Informationen zwischen STACK, Tutor-Workflow, symbolischem Prüfsystem und LLM,
- prototypische Anbindung von Maxima beziehungsweise Headless STACK zur mathematischen Prüfung von Antworten oder generierten Ausdrücken,
- Untersuchung einer optionalen Werkzeug- beziehungsweise Agentenschnittstelle, über die das LLM definierte mathematische Prüfoperationen anfordern kann,
- Orchestrierung des Tutor-Workflows, beispielsweise mit n8n,
- Auswahl und Erprobung mindestens eines selbst gehosteten LLMs,
- Entwicklung eines Prompt-Workflows mit gestuften Hilfen und expliziten Regeln zur Vermeidung von Lösungsverrat,
- Vergleich unterschiedlicher Kontext- und Diagnosebedingungen,

◀3 Zielsetzung

- technische Evaluation anhand eines kuratierten Aufgabensatzes mit typischen Fehlerantworten,
- optional Einbeziehung anonymisierter realer Studierendenantworten, sofern diese rechtlich und organisatorisch verfügbar sind,
- optional explorative Erprobung mit Studierenden mittels Think-Aloud-Methode,
- Ableitung von Gestaltungsempfehlungen für LLM-basierte Mathematik-tutoren.

Die Arbeit ist ausdrücklich als prototypische Untersuchung angelegt. Eine vollständige produktive Integration in Moodle, eine umfassende Rechteverwaltung, eine groSS angelegte empirische Wirksamkeitsstudie und die Abdeckung beliebiger mathematischer Themen sind nicht Gegenstand der Arbeit.

◀ 4 Untersuchungsgegenstand und Eingrenzung

Untersuchungsgegenstand ist ein selbst gehosteter LLM-Tutor für digitale Mathematikaufgaben, der mit einem symbolischen mathematischen Prüfsystem verbunden wird.

Die Eingrenzung erfolgt auf:

- mathematisches Thema: Differentialrechnung,
- Aufgabentypen: ausgewählte Aufgaben zu grundlegenden Ableitungsregeln,
- mathematische Prüfung: STACK/Maxima sowie optional Headless STACK,
- Modellbetrieb: selbst gehostetes LLM,
- technische Orchestrierung: prototypischer Workflow, beispielsweise über n8n,
- didaktischer Fokus: adaptive und gestufte Hinweise ohne unmittelbare Lösungsvorgabe,
- Evaluationsfokus: technische Funktionsfähigkeit und Qualität der Tutorhinweise.

Nicht betrachtet werden:

- ein vollständiges produktives Moodle-Plugin,
- eine allgemeine Tutoring-Plattform für beliebige mathematische Fachgebiete,
- die alleinige mathematische Bewertung durch das LLM,
- umfangreiche Learning-Analytics-Funktionen,
- eine groSS angelegte quantitative Wirksamkeitsstudie,

◀4 Untersuchungsgegenstand und Eingrenzung

- die Implementierung einer umfangreichen embedding-basierten Vektordatenbank.

Auf eine umfangreiche Vektordatenbank wird im Prototyp voraussichtlich verzichtet. Da der Untersuchungsbereich auf einen kuratierten Satz von Aufgaben aus der Differentialrechnung beschränkt ist und mit Aufgaben-ID, Aufgabentyp, Diagnosecode und Hilfestufe bereits strukturierte Auswahlmerkmale vorliegen, kann der relevante Kontext deterministisch aus einer datei- oder datenbankbasierten Wissensbasis geladen werden. Eine semantische Vektorsuche wird als mögliche Erweiterung für gröSSere und heterogene Aufgabenbestände diskutiert.

◀ 5 Methodisches Vorgehen

Die Arbeit folgt einem entwicklungsorientierten Forschungsansatz. Im Mittelpunkt stehen die Konzeption, Implementierung und Evaluation eines technischen Artefakts. Das Vorgehen umfasst eine Literatur- und Marktanalyse, eine Anforderungsanalyse, die prototypische Softwareentwicklung sowie eine technische und didaktisch-qualitative Evaluation.

5.1 Literatur- und Marktanalyse

Zu Beginn wird der wissenschaftliche und technische Stand analysiert. Die Literaturrecherche umfasst insbesondere:

- STACK und computeralgebrabasiertes E-Assessment,
- intelligente Tutorsysteme und automatische Hint-Generierung,
- LLM-basierte Mathematikutoren,
- Retrieval-Augmented Generation und deterministische Kontextanreicherung,
- neuro-symbolische Systeme und symbolische Solver-Rückkopplung,
- Prompt-Design und gestufte Tutorhinweise,
- formative Rückmeldung und Vermeidung von Lösungsverrat,
- Think-Aloud als qualitative Evaluationsmethode.

Ergänzend werden bestehende kommerzielle und institutionelle Angebote betrachtet. Dazu zählen allgemeine LLM-Systeme, mathematikspezifische Solver, adaptive Lernplattformen sowie hochschulbezogene KI-Tutoren. Als möglicher institutioneller Referenzpunkt wird unter anderem das Angebot *KI & ME* der Hochschule Magdeburg-Stendal beziehungsweise die Plattform <https://kiandme.h2.de> geprüft. Ziel ist eine Einordnung der geplanten Lösung und keine umfassende Marktstudie.

◀5 Methodisches Vorgehen

5.2 Analyse der Anforderungen

Auf Grundlage der Literatur- und Marktanalyse werden die funktionalen, didaktischen und technischen Anforderungen an den Prototyp festgelegt. Betrachtet werden insbesondere folgende Fragen:

- Welche Informationen können aus STACK oder einer vergleichbaren digitalen Aufgabe bereitgestellt werden?
- Welche Informationen benötigt das LLM, um einen hilfreichen Hinweis zu erzeugen?
- Welche mathematischen Prüfoperationen müssen durch Maxima oder Headless STACK bereitgestellt werden?
- Welche Informationen dürfen auf den jeweiligen Hilfestufen verwendet werden?
- Wie soll der Tutor reagieren, wenn keine verlässliche Diagnose vorliegt?
- Wie werden fehlerhafte, manipulierte oder unzureichende Eingaben behandelt?
- Welche Anforderungen bestehen an Datenschutz, Nachvollziehbarkeit und Antwortzeit?

Das Ergebnis ist ein Anforderungskatalog für Systemarchitektur, Tutor-Workflow und Evaluation.

5.3 Auswahl und Modellierung des Aufgabensatzes

Für die Evaluation wird ein begrenzter Aufgabensatz aus der Differentialrechnung zusammengestellt. Dieser soll unterschiedliche Ableitungsregeln und typische Fehlerklassen abdecken, beispielsweise:

- Potenzregel,
- Produktregel,
- Quotientenregel,
- Kettenregel,
- Ableitung von Exponential- und trigonometrischen Funktionen,
- Kombination mehrerer Ableitungsregeln.

◀5 Methodisches Vorgehen

Zu jeder Aufgabe werden typische fehlerhafte Antworten definiert. Diese können aus didaktischer Literatur, der Erfahrung von Lehrenden, vorhandenen PRT-Strukturen und, sofern verfügbar, anonymisierten realen Studierendenantworten abgeleitet werden.

Eine exemplarische Aufgabe lautet:

$$f(x) = -5e^{x^2-2e^x}.$$

Mögliche Fehler sind:

- fehlende innere Ableitung,
- fehlerhafte Ableitung von e^x ,
- fehlerhafte Ableitung von x^2 ,
- vergessener konstanter Faktor,
- Vorzeichenfehler,
- Ableitung ausschließSSlich des inneren Exponenten.

In einer STACK-Aufgabe werden solche Fehlermuster durch die Autorinnen und Autoren im PRT modelliert. Der PRT identifiziert die Fehler anschließend durch symbolische Tests. Optional wird untersucht, inwiefern eine dynamische Analyse mit Maxima oder Headless STACK vergleichbare beziehungsweise ergänzende Informationen erzeugen kann.

5.4 Entwicklung des technischen Prototyps

Der Prototyp soll folgende Informations- und Verarbeitungskette realisieren:

digitale Mathematikaufgabe → Tutor-Workflow
→ STACK/Maxima oder Headless STACK
→ selbst gehostetes LLM
→ Tutor-Interface.

Für die prototypische Moodle-STACK-Anbindung kann ein Feedback-Link verwendet werden, der eine Aufgaben-ID, die Studierendenantwort und gegebenenfalls einen PRT-Diagnosecode an das Tutor-Interface übergibt. Eine direkte Rückintegration des Hinweises in die ursprüngliche STACK-Oberfläche ist optional und nicht zwingende Voraussetzung des Minimalprototyps.

Der Prototyp soll mindestens folgende Funktionen bereitstellen:

- Entgegennahme einer Aufgaben-ID und Studierendenantwort,

◀5 Methodisches Vorgehen

- Entgegennahme oder Ermittlung eines Diagnosecodes,
- Validierung der übertragenen Daten,
- Laden aufgabenspezifischer Kontextinformationen,
- Aufruf von Maxima beziehungsweise Headless STACK zur mathematischen Prüfung,
- Erzeugung eines strukturierten Tutor-Prompts,
- Aufruf eines selbst gehosteten LLMs,
- Anzeige oder Rückgabe des generierten Hinweises,
- Protokollierung der für die Evaluation erforderlichen technischen Daten.

5.5 Anbindung von Maxima und Headless STACK

Maxima beziehungsweise Headless STACK soll als deterministische mathematische Komponente in den Tutor-Workflow eingebunden werden. Untersucht werden können folgende Varianten:

1. PRT-basierte Diagnose

STACK bewertet die Antwort und übergibt einen durch den PRT bestimmten Diagnosecode an den Tutor.

2. Direkter Maxima-Aufruf

Der Workflow übermittelt mathematische Ausdrücke an eine Maxima-Schnittstelle und erhält beispielsweise Ergebnisse zu Äquivalenz, Differenz, Vereinfachung oder Ableitung.

3. Headless-STACK-Prüfung

Eine STACK-Prüfumgebung wird ohne Moodle-Oberfläche aufgerufen, um vorhandene STACK-Tests und PRT-Logik wiederzuverwenden.

4. Werkzeugnutzung durch einen Agenten

Das LLM kann innerhalb eines streng begrenzten Workflows definierte mathematische Prüfwerkzeuge anfordern. Die Werkzeugergebnisse werden anschließend für die Formulierung des Hinweises verwendet.

Die agentische Variante wird nur umgesetzt, sofern sie innerhalb des zeitlichen Rahmens einen erkennbaren Mehrwert gegenüber einem deterministischen Workflow bietet. Das LLM erhält keine uneingeschränkte Ausführungs- oder Bewertungsbefugnis.

◀5 Methodisches Vorgehen

5.6 Workflow-Orchestrierung mit n8n

Die technische Prozesssteuerung kann mit *n8n* prototypisch umgesetzt werden. Ein möglicher Workflow umfasst:

1. Entgegennahme der Anfrage aus STACK,
2. Prüfung und Normalisierung der Eingabeparameter,
3. Laden der Aufgaben- und Tutor-Metadaten,
4. Aufruf von Maxima oder Headless STACK,
5. Auswahl der Hilfestufe und des zugehörigen Prompt-Workflows,
6. Aufruf des selbst gehosteten LLMs,
7. optionale mathematische oder regelbasierte Prüfung der LLM-Ausgabe,
8. Rückgabe des Tutorhinweises,
9. Speicherung technischer Messwerte für die Evaluation.

Dabei ist zu prüfen, ob *n8n* als vollständige Orchestrierungsschicht verwendet wird oder ob einzelne Funktionen in einer ergänzenden Python-Anwendung implementiert werden. Die Entscheidung erfolgt anhand von Nachvollziehbarkeit, Wartbarkeit, Funktionsumfang und Implementierungsaufwand.

5.7 Kontext- und Wissensbasis

Die Wissensbasis wird für den begrenzten Aufgabensatz kuratiert und kann folgende Bestandteile enthalten:

- Aufgabentext und Aufgaben-ID,
- mathematisches Thema und Lernziel,
- relevante Ableitungsregeln,
- Musterlösung und geprüfte Zwischenschritte,
- typische Fehler und PRT-Diagnosecodes,
- geeignete Tutorstrategien,
- Regeln für die einzelnen Hilfestufen,
- zulässige und unzulässige Inhalte eines Hinweises.

◀5 Methodisches Vorgehen

Die Kontextauswahl erfolgt voraussichtlich deterministisch anhand von Aufgaben-ID, Diagnosecode und Hilfestufe. Dieser Ansatz reduziert technische Komplexität und vermeidet zusätzliche Retrieval-Fehler bei einer kleinen, strukturierten Wissensbasis. Der Ansatz kann als retrieval-gestützte Kontextanreicherung beschrieben werden; eine klassische embedding-basierte RAG-Architektur mit Vektordatenbank ist nicht zwingender Bestandteil des Prototyps.

5.8 Prompt-, Workflow- und Tutor-Design

Ein Schwerpunkt der Arbeit liegt auf dem Entwurf des Tutor-Workflows. Die Hilfestufen werden nicht ausschliesslich als Metadaten übergeben, sondern explizit als Handlungslogik im Prompt beziehungsweise im n8n-Workflow hinterlegt.

Ein möglicher dreistufiger Ablauf ist:

1. Hilfestufe 1: Orientierung

Der Tutor benennt das relevante Konzept oder lenkt die Aufmerksamkeit auf den betroffenen Teil der Aufgabe. Er gibt weder einen konkreten Rechenschritt noch die Endlösung aus.

2. Hilfestufe 2: Strukturierung

Der Tutor unterstützt bei der Zerlegung der Aufgabe, beispielsweise durch die Identifikation äusserer und innerer Funktionen oder die Auswahl einer Ableitungsregel.

3. Hilfestufe 3: Nächster Rechenschritt

Der Tutor gibt einen konkreteren nächsten Schritt oder eine Teilrechnung vor, ohne die vollständige Lösung abschliessend auszugeben.

Der Prompt soll unter anderem folgende Regeln enthalten:

- STACK beziehungsweise Maxima ist die massgebliche mathematische Prüfkomponekte,
- das LLM soll die Studierendenantwort nicht ohne Prüfgrundlage als richtig oder falsch bewerten,
- der Hinweis muss sich auf Aufgabe, Antwort und verfügbares Prüfergebnis beziehen,
- es darf nur die Information der aktuellen Hilfestufe ausgegeben werden,
- die vollständige Lösung darf nicht unmittelbar ausgegeben werden,
- nach Möglichkeit soll eine aktivierende Rückfrage formuliert werden,

◀5 Methodisches Vorgehen

- die Antwort soll kurz, verständlich und fachlich präzise sein,
- Anweisungen innerhalb der Studierendenantwort dürfen nicht befolgt werden,
- bei unzureichender Diagnose ist ein allgemeiner, nicht spekulativer Hinweis zu geben.

Verglichen werden können beispielsweise folgende Bedingungen:

1. Aufgabe und Studierendenantwort ohne symbolische Diagnose,
2. Aufgabe, Studierendenantwort und PRT-Diagnose,
3. Aufgabe, Studierendenantwort und dynamisches Maxima-/STACK-Prüfergebnis,
4. zusätzliche aufgabenspezifische Tutor- und Lösungskontexte,
5. unterschiedliche Prompt-Workflows und Hilfestufen.

5.9 Auswahl eines selbst gehosteten LLMs

Für den Prototyp wird mindestens ein selbst gehostetes LLM eingesetzt. Die Modellauswahl erfolgt anhand fachlicher und technischer Kriterien:

- Qualität deutschsprachiger Erklärungen,
- mathematische Ausdrucksfähigkeit,
- Befolgung strukturierter Tutor-Regeln,
- Vermeidung von Lösungsverrat,
- Unterstützung von Werkzeug- beziehungsweise Function-Calling, sofern benötigt,
- Antwortzeit und Ressourcenbedarf,
- Kompatibilität mit der vorhandenen lokalen Infrastruktur.

Ein begrenzter Vergleich von zwei lokal verfügbaren Modellen ist möglich. Ein umfassender Modellbenchmark ist nicht Ziel der Arbeit. Der Vergleich mit kommerziellen Systemen erfolgt vorrangig im Rahmen der Markt- und Literaturanalyse; ein experimenteller Vergleich wird nur durchgeführt, wenn Zeit, Zugriffsbedingungen und Datenschutzerfordernissen dies zulassen.

◀5 Methodisches Vorgehen

5.10 Evaluation

Die Evaluation besteht aus einer technischen sowie einer fachlich-didaktischen Komponente.

Technische Evaluation

Für die technische Evaluation wird ein Satz von Differentialrechnungsaufgaben mit typischen korrekten und fehlerhaften Antworten zusammengestellt. Soweit verfügbar und datenschutzrechtlich zulässig, können anonymisierte reale Studierendenantworten einbezogen werden. Andernfalls werden plausible Fehlerantworten auf Grundlage von PRT-Strukturen, Lehrendenerfahrung und Literatur konstruiert.

Geprüft werden insbesondere:

- korrekte Übergabe der Informationen aus STACK,
- korrekte Kodierung mathematischer Eingaben,
- korrekte Zuordnung von Aufgaben-ID und Kontext,
- korrekte Erkennung beziehungsweise Übergabe des PRT-Diagnosecodes,
- Funktionsfähigkeit der Maxima- oder Headless-STACK-Anbindung,
- Übereinstimmung symbolischer Prüfergebnisse mit erwarteten Ergebnissen,
- korrekte Ausführung des n8n-Workflows,
- korrekte Auswahl der Hilfestufe,
- erfolgreiche Kommunikation mit dem selbst gehosteten LLM,
- Stabilität bei syntaktisch fehlerhaften und unerwarteten Eingaben,
- Antwortzeiten der einzelnen Systemkomponenten und des Gesamtworkflows.

Optional wird verglichen, wie zuverlässig typische Fehler durch vorab modellierte PRT-Zweige und durch eine dynamische Maxima-/Headless-STACK-Analyse erkannt werden.

◀5 Methodisches Vorgehen

Fachlich-didaktische Evaluation

Die generierten Tutorhinweise werden anhand eines Kriterienrasters bewertet. Mögliche Kriterien sind:

- fachliche Korrektheit,
- Übereinstimmung mit dem symbolischen Prüfergebnis,
- Passung zur Studierendenantwort,
- Passung zum PRT-Diagnosecode beziehungsweise Fehlerfall,
- Verständlichkeit,
- Hilfreichkeit für den nächsten Lösungsschritt,
- Angemessenheit der Hilfestufe,
- Vermeidung von Lösungsverrat,
- Aktivierung der Lernenden durch Rückfragen,
- sprachliche Präzision.

Die Bewertung kann durch Lehrende oder Tutorinnen und Tutoren erfolgen. Für zentrale Kriterien können sowohl Likert-Skalen als auch binäre Merkmale verwendet werden, beispielsweise:

- enthält eine mathematisch falsche Aussage: ja/nein,
- enthält die vollständige Lösung: ja/nein,
- bezieht sich auf den diagnostizierten Fehler: ja/nein,
- entspricht der vorgesehenen Hilfestufe: ja/nein.

Optionale Think-Aloud-Erprobung

Optional kann eine kleine explorative Erprobung mit Studierenden durchgeführt werden. Hierfür eignet sich die **Think-Aloud-Methode**. Dabei bearbeiten Studierende ausgewählte Aufgaben, erhalten generierte Hinweise und verbalisieren ihre Gedanken während der weiteren Bearbeitung.

Mögliche Leitfragen sind:

- War der Hinweis verständlich?
- Hat der Hinweis geholfen, den eigenen Fehler zu erkennen?
- Unterstützte der Hinweis einen eigenständigen nächsten Schritt?

◀5 Methodisches Vorgehen

- War der Hinweis zu allgemein, angemessen oder zu direkt?
- Wurde die Lösung vorweggenommen?
- Würde ein solcher Tutor beim selbstständigen Üben als hilfreich empfunden?

Die qualitative Untersuchung dient nicht dem Nachweis allgemeiner Lernwirksamkeit, sondern der explorativen Einschätzung von Verständlichkeit, Nützlichkeit und didaktischer Angemessenheit.

◀ 6 Erwartete Ergebnisse und wissenschaftlicher Beitrag

Als Ergebnis der Arbeit wird ein funktionsfähiger Prototyp erwartet, der eine digitale Mathematikaufgabe mit einem symbolischen Prüfsystem und einem selbst gehosteten LLM-Tutor verbindet. Zusätzlich sollen ein definierter Tutor-Workflow, Prompt-Templates, gestufte Hilfen, eine kuratierte Aufgaben- und Wissensbasis sowie ein Evaluationsdatensatz entstehen.

Der wissenschaftliche Beitrag liegt insbesondere in der systematischen Untersuchung der Aufgabenteilung zwischen symbolischer Mathematik und generativer Sprachverarbeitung. Während STACK beziehungsweise Maxima die mathematische Prüfung übernehmen, soll das LLM didaktische Hinweise auf Grundlage kontrollierter Prüfergebnisse formulieren.

Der erwartete Beitrag umfasst:

1. ein prototypisches Integrationskonzept für digitale Mathematikaufgaben, symbolische Prüfung und LLM-Tutoring,
2. eine Untersuchung der Anbindung von Maxima beziehungsweise Headless STACK an einen Tutor-Workflow,
3. einen Vergleich zwischen PRT-basierter Fehlerdiagnose und optionaler dynamischer symbolischer Analyse,
4. einen Prompt- und Workflow-Ansatz für gestufte Hilfen,
5. eine Bewertung selbst gehosteter LLMs für den betrachteten Tutor-Anwendungsfall,
6. eine technische und fachlich-didaktische Evaluation anhand typischer Fehlerantworten,
7. Gestaltungsempfehlungen für zukünftige LLM-basierte Mathematiktutoren.

◀ 7 Quellenlage / aktueller Stand der Technik

Die Arbeit liegt an der Schnittstelle von mathematischem E-Assessment, intelligenten Tutorsystemen, generativer KI, neuro-symbolischen Systemen und formativem Feedback. Die Quellenlage umfasst daher sowohl etablierte Forschung zu automatisierter mathematischer Bewertung und Tutorhinweisen als auch aktuelle Arbeiten zu LLM-basierten Mathematiktutoren, Retrieval-Augmented Generation und symbolischer Verifikation.

7.1 Mathematisches E-Assessment und STACK

STACK ermöglicht die symbolische Bewertung mathematischer Antworten mit Maxima. Potential Response Trees erlauben es, typische Fehlermuster durch von Lehrenden definierte symbolische Tests zu identifizieren und differenziertes Feedback auszugeben. Die Forschung zu STACK hebt die mathematische Ausdrucksstärke und die Möglichkeiten formativer Rückmeldung hervor, verweist jedoch zugleich auf den hohen Aufwand für Aufgabenerstellung, PRT-Modellierung und Pflege der Feedbacklogik.

Für die geplante Arbeit ist STACK sowohl als Ausgangssystem für digitale Aufgaben als auch als mögliche Headless-Prüfkomponente relevant. Zu untersuchen ist, wie vorhandene PRT-Diagnosen für einen LLM-Tutor nutzbar gemacht werden können und ob eine zusätzliche dynamische Anbindung von Maxima oder Headless STACK einen Mehrwert bietet.

7.2 LLM-basierte Mathematiktutoren und Hint-Generierung

Aktuelle Arbeiten untersuchen LLMs zur mathematischen Fragebeantwortung, zur Erzeugung von Feedback und zur Generierung gestufter Hinweise. Eine zentrale Herausforderung besteht darin, Lernende zu unterstützen, ohne die vollständige Lösung vorwegzunehmen. Aus der Forschung zu intelligenten Tutorsystemen sind Hint-Sequenzen bekannt, bei denen Hinweise schrittweise konkreter werden. Diese Ansätze bilden eine Grundlage für den in dieser Arbeit geplanten Prompt-Workflow.

◀7 Quellenlage / aktueller Stand der Technik

Forschung zu LLM-generierten Mathematikhinweisen zeigt zugleich, dass solche Hinweise nicht durchgängig zuverlässig genug sind, um ungeprüft an Lernende ausgegeben zu werden. Lehrendenkontrolle, symbolische Verifikation, strukturierte Prompt-Regeln und aufgabenspezifischer Kontext werden daher als wichtige Absicherungen betrachtet.

7.3 Retrieval-Augmented Generation und Kontextsteuerung

Retrieval-Augmented Generation wird eingesetzt, um LLM-Antworten in geprüften externen Materialien zu verankern. Arbeiten zur mathematischen Fragebeantwortung zeigen, dass RAG die fachliche und curriculare Verankerung verbessern kann. Zugleich besteht ein Zielkonflikt zwischen starker Bindung an abgerufene Materialien und der sprachlichen beziehungsweise didaktischen Qualität der Antwort. Eine hohe Groundedness ist nicht automatisch mit hoher Korrektheit oder Hilfreichkeit gleichzusetzen.

Für den begrenzten Prototyp ist keine umfangreiche Vektordatenbank vorgesehen. Stattdessen sollen Aufgaben-ID, Diagnosecode und Hilfestufe zur deterministischen Auswahl kuratierter Kontextinformationen verwendet werden. Dieser Ansatz ermöglicht eine kontrollierbare und reproduzierbare Kontextanreicherung. Embedding-basiertes Retrieval bleibt eine mögliche Erweiterung für gröSSere Aufgabenbestände.

7.4 Symbolische Verifikation und neuro-symbolische Ansätze

Aktuelle Arbeiten kombinieren LLMs und RAG bereits mit Computer-Algebra-Systemen oder symbolischen Solvern. Dabei werden generierte Rechenschritte oder Endergebnisse beispielsweise durch SymPy oder Maxima geprüft. Solche Arbeiten zeigen, dass Retrieval allein symbolische Ausführungsfehler nicht zuverlässig verhindert und eine deterministische mathematische Kontrollkomponente sinnvoll ist.

Die geplante Arbeit knüpft an diese neuro-symbolischen Ansätze an, setzt jedoch einen anderen Schwerpunkt. Das symbolische System soll nicht nur nachträglich eine vom LLM erzeugte Lösung prüfen. Vielmehr sollen STACK, PRTs, Maxima oder Headless STACK die Studierendenantwort bewerten beziehungsweise diagnostizieren und dem LLM eine kontrollierte Grundlage für die didaktische Hinweisgenerierung liefern.

7.5 Kommerzielle und institutionelle Systeme

Es existieren verschiedene Systeme zur mathematischen Unterstützung:

- allgemeine LLM-basierte Chatbots wie *ChatGPT*, *Claude*, *Gemini*, *Microsoft Copilot* und *Perplexity*,

◀7 Quellenlage / aktueller Stand der Technik

- mathematikspezifische Tutor- und Solver-Systeme wie *Photomath*, *Symbolab*, *Mathway*, *WolframAlpha Pro* und *Microsoft Math Solver*,
- E-Assessment-Systeme wie *Möbius*, *ALEKS*, *MyMathLab*, *WirisQuizzes*, *STACK* und *WeBWorK*,
- adaptive Tutoring-Systeme und Lernplattformen wie *Khanmigo*, *Carnegie Learning/MATHia*, *Squirrel AI* und *Querium*,
- institutionelle Hochschulangebote wie gegebenenfalls *KI & ME* der Hochschule Magdeburg-Stendal.

Diese Systeme decken Teilaspekte wie Lösungserzeugung, symbolische Berechnung, adaptive Lernpfade, automatische Bewertung oder dialogische Unterstützung ab. Eine transparente und selbst gehostete Kombination aus STACK-/Maxima-Prüfung, explizitem Tutor-Workflow und didaktisch kontrollierter LLM-Hinweisgenerierung ist jedoch nicht als allgemeine Standardlösung etabliert.

7.6 Forschungslücke

Die allgemeine Kombination von LLMs, RAG und symbolischen Solvern ist bereits Gegenstand aktueller Forschung. Die Forschungslücke kann daher nicht in der erstmaligen Verbindung dieser Technologien gesehen werden.

Weniger untersucht ist hingegen die gezielte Nutzung einer symbolischen Bewertung oder Fehlerdiagnose von Studierendenantworten zur Steuerung eines didaktisch ausgerichteten LLM-Tutors. Insbesondere die Verbindung von:

- digitalen Mathematikaufgaben,
- PRT-basierten Diagnosecodes,
- Maxima oder Headless STACK als mathematischer Prüfinstanz,
- einem selbst gehosteten LLM,
- einem expliziten Tutor-Workflow mit Hilfestufen,
- und der systematischen Vermeidung von Lösungsverrat

ist bislang nur unzureichend untersucht. Genau diese Kopplung bildet den zentralen Untersuchungsgegenstand der Arbeit.

8 Zeitplan

Ein möglicher Zeitplan für eine Bearbeitungsdauer von 19 Wochen ist in Tabelle 8.1 dargestellt.

Die schriftliche Dokumentation erfolgt parallel zur Entwicklung und Evaluation. Falls einzelne optionale Bestandteile, insbesondere die Agentenanbindung oder die Think-Aloud-Erprobung, nicht innerhalb des Zeitrahmens realisierbar sind, wird der Schwerpunkt auf den deterministischen Workflow und die technische sowie fachlich-didaktische Evaluation gelegt.

◀8 Zeitplan

Tabelle 8.1: Vorläufiger Zeitplan

Phase		Zeitraum	Inhalt
Literatur- und Marktanalyse		Woche 1–3	Recherche zu STACK, Maxima, Headless STACK, LLM-Tutoring, symbolischer Verifikation, Prompt-Design, kommerziellen und institutionellen Systemen
Anforderungsanalyse und Eingrenzung		Woche 4	Präzisierung der Architektur, Evaluationsfragen, Aufgabentypen und Mindestanforderungen
Aufgaben- und Fehlerkorpus		Woche 5–6	Auswahl von Aufgaben aus der Differentialrechnung; Definition typischer Fehler; Prüfung der Verfügbarkeit anonymisierter realer Studierendenantworten
Technische Basisintegration		Woche 7–8	Informationsübergabe aus STACK; Aufbau des Tutor-Interfaces; Validierung der Parameter und Kontextverwaltung
Maxima-/STACK-Anbindung		Woche 9–10	Implementierung und Test einer Maxima-beziehungsweise Headless-STACK-Schnittstelle; optionaler Vergleich mit PRT-Diagnosen
n8n- und LLM-Workflow		Woche 11–12	Orchestrierung des Workflows; Anbindung des selbst gehosteten LLMs; Fehlerbehandlung und Protokollierung
Prompt- und Tutor-Design		Woche 13–14	Entwicklung und Erprobung des gestuften Prompt-Workflows; Regeln gegen Lösungsverrat; Auswahl des Modells
Technische Evaluation		Woche 15–16	Tests mit Aufgaben und typischen Fehlerantworten; Messung von Prüfergebnissen, Stabilität und Antwortzeiten
Didaktische Evaluation		Woche 17	Expertenbewertung und, sofern realisierbar, explorative Think-Aloud-Erprobung
Auswertung und Diskussion		Woche 18	Auswertung der Ergebnisse; Beantwortung der Forschungsfragen; Ableitung von Gestaltungsempfehlungen
Abschluss		Woche 19	Fertigstellung, Überarbeitung, formale Prüfung und Abgabe der Arbeit

◀ 9 Vorläufige Gliederung

1. Einleitung

- a) Motivation und Problemstellung
- b) Zielsetzung
- c) Forschungsfragen
- d) Abgrenzung
- e) Aufbau der Arbeit

2. Theoretische und technische Grundlagen

- a) Digitale Mathematikaufgaben und E-Assessment
- b) Moodle, STACK und Potential Response Trees
- c) Maxima und Headless STACK
- d) Large Language Models und selbst gehostete Modellinferenz
- e) LLM-basierte Mathematikutoren und Hint-Generierung
- f) Retrieval-Augmented Generation und Kontextanreicherung
- g) Neuro-symbolische Systeme und symbolische Verifikation
- h) Formatives Feedback und gestufte Hilfen

3. Stand der Forschung und Technik

- a) Wissenschaftliche Arbeiten zu LLM-Tutoring
- b) RAG für mathematische Lernunterstützung
- c) Symbolische Solver in LLM-Systemen
- d) Kommerzielle und institutionelle Angebote
- e) Forschungslücke

4. Anforderungsanalyse und Konzeption

- a) Fachliche Eingrenzung auf Differentialrechnung
- b) Funktionale und didaktische Anforderungen

◀9 Vorläufige Gliederung

- c) Systemarchitektur und Datenfluss
- d) Aufgaben- und Kontextmodell
- e) PRT-basierte und dynamische Diagnose
- f) Tutor-Workflow und Hilfestufen
- g) Sicherheits- und Datenschutzaspekte

5. Prototypische Implementierung

- a) STACK-Anbindung und Parameterübergabe
- b) Tutor-Interface
- c) Anbindung von Maxima beziehungsweise Headless STACK
- d) n8n-Workflow
- e) Anbindung des selbst gehosteten LLMs
- f) Prompt- und Tutor-Policy
- g) Fehlerbehandlung und Protokollierung

6. Evaluation

- a) Evaluationsdesign und Aufgabensatz
- b) Typische und reale Studierendenfehler
- c) Technische Evaluation
- d) Vergleich der Diagnoseansätze
- e) Fachlich-didaktische Bewertung
- f) Optionale Think-Aloud-Erprobung

7. Ergebnisse und Diskussion

- a) Ergebnisse der technischen Umsetzung
- b) Qualität und Zuverlässigkeit der mathematischen Prüfung
- c) Qualität der generierten Tutorhinweise
- d) Einhaltung der Hilfestufen und Lösungserrat
- e) Eignung des selbst gehosteten LLMs
- f) Grenzen des Prototyps

8. Fazit und Ausblick

- a) Beantwortung der Forschungsfragen
- b) Zentrale Erkenntnisse
- c) Gestaltungsempfehlungen
- d) Ausblick auf produktive Moodle-Integration, gröSSere Aufgabenbestände und weiterführende Agentenarchitekturen